Analog-to-Digital Converter (ADC)

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



- Adapted from
- ***Richard Kuo***
- ***Assistant Professor***

1

ADC pins

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

Based on NUC140 LQFP 100pin

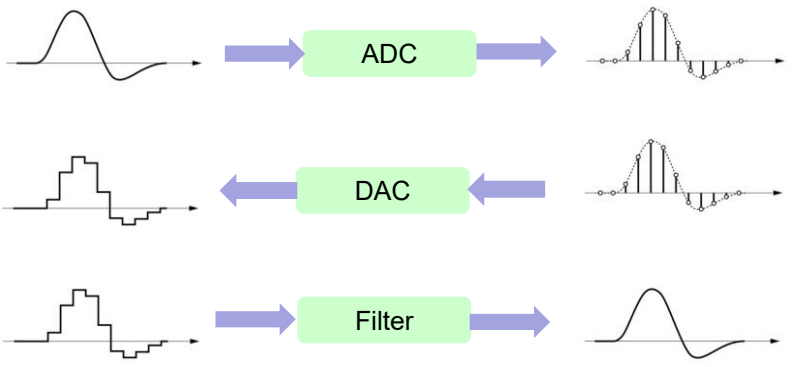
- ADC0 / GPA0 : pin 71
- ADC1 / GPA1 : pin 72
- ADC2 / GPA2 : pin 73
- ADC3 / GPA3 : pin 74
- ADC4 / GPA4 : pin 75
- ADC5 / GPA5 : pin 76
- ADC6 / GPA6 : pin 77
- ADC7 / GPA7 : pin 78



3

Signal Conversion

Department of Electrical and Electronic Engineering
Universiti Teknologi PETRONAS



4

Digital Conversion

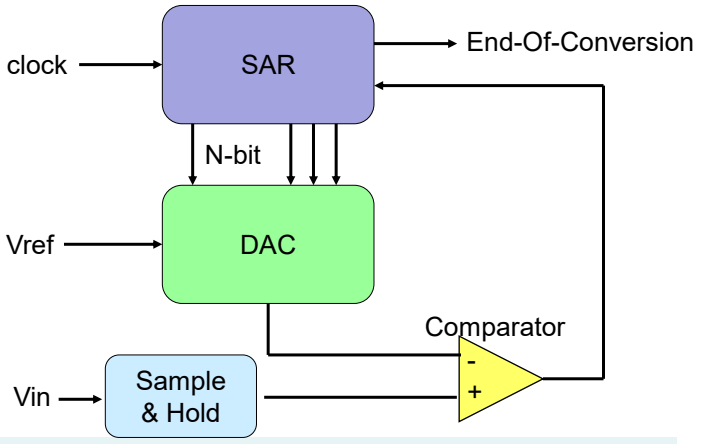
Department of Electrical and Electronic Engineering
Universiti Teknologi PETRONAS

- Vin is input voltage for ADC to sample, connected to a comparator
- Vref is reference voltage for ADC to compare with Vin
  - Vref is connected to 3.3V on NUC140 learning board
- ADC output is a 12-bit binary code, so its digital range 0 ~ 4095
- The digital value correspond to a voltage level of Vin
  - 0 : 0000_0000_0000 is 3.3V * 0/4096 = 0 V
  - 1 : 0000_0000_0001 is 3.3V * 1/4096
  - 2048 : 1000_0000_0000 is 3.3V * 2048/4096 = 1/2 Vref
  - 4095 : 1111_1111_1111 is 3.3V * 4095/4096 = 1 Vref

6

SAR (Successive Approximation Register) ADC

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



7

ADC specification

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

- 12-bit SAR ADC
- Analog input voltage range:
  - 0~Vref (Max to 5.0V).
- Operation voltage:
  - AVDD=3.0V~5.5V;
- Input channel:
  - Up to 8 single-end analog input channels
  - 4 pairs of differential analog input channel.
- Up to 700~800KHz conversion rate.
- The maximum ADC operating frequency is 16M Hz
- Three operation modes
  - **Single mode**
  - **Single-cycle mode**
  - **Continuous scan mode**

8

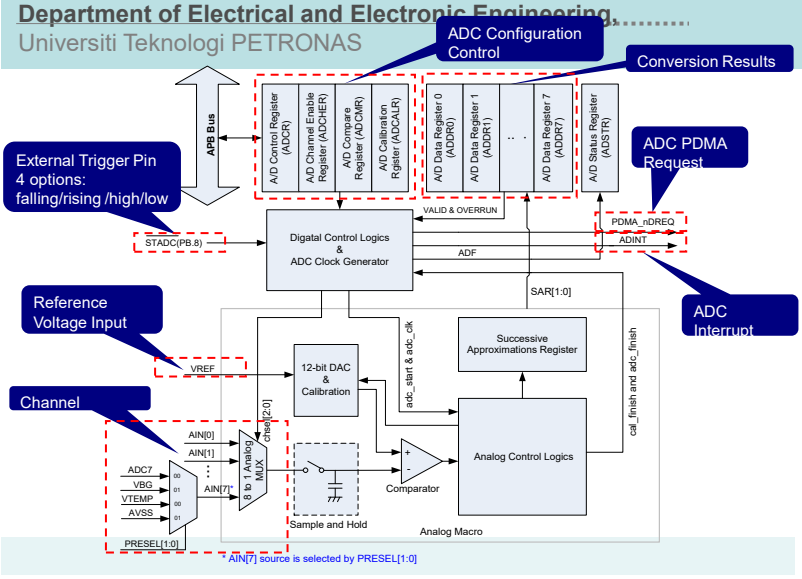
ADC pins configuration

- The ADC input pins share with GPIO port A
- The ADC input pins must be configured in **input type**
- Single-end input mode
  - Channel 0 ~ Channel 7 share with GPA0~GPA7
- Differential input mode
  - Channel 0 ~ Channel 3
  - Differential input channels are the paired channels

Differential input paired channel	Single-end input channel	
0	0	1
1	2	3
2	4	5
3	6	7

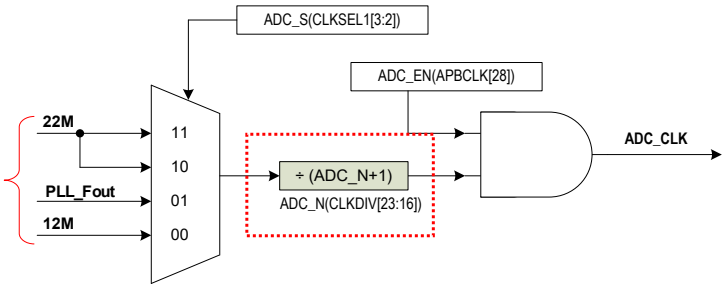


NuMicro ADC block diagram



NuMicro ADC clock source

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

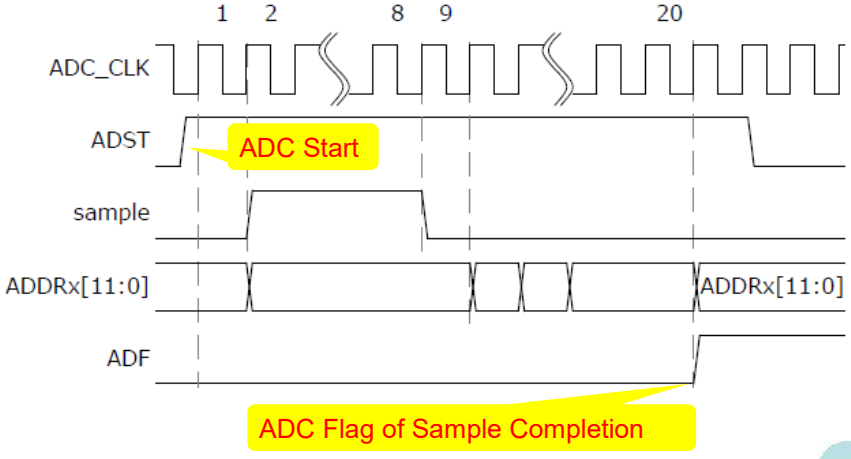


- The maximum ADC clock frequency is 16MHz
- The ADC clock frequency = (ADC clock source frequency) / (ADC_N+1);
  ADC_N is an 8-bit pre-scaler

11

ADC Single Mode Conversion

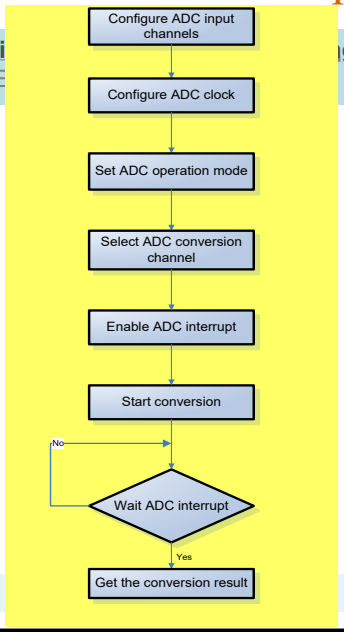
Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



12

Control Flow of ADC interrupt

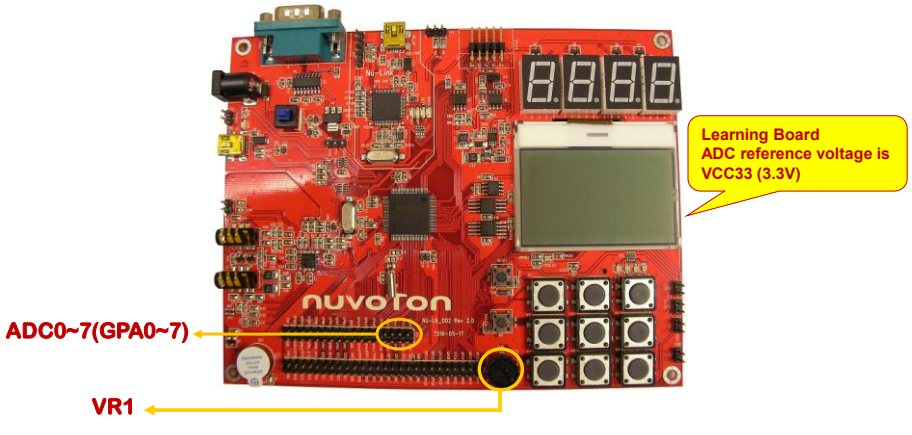
Department of Electri…ng…
Universiti Teknologi PE…



15

NuMicro MCU Learning Board

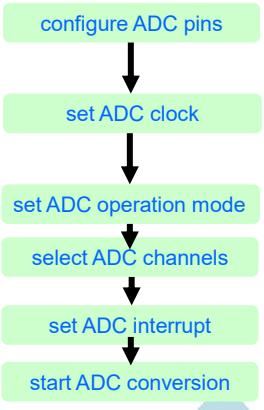
Department of Electrical and Electronic Engineering
Universiti Teknologi PETRONAS



16

InitADC() : using ADC7


```
void InitADC(void) {
/* Step 1. GPIO initial */
GPIOA->OFFD|=0x00800000;          //Disable digital input path
SYS->GPAMFP.ADC7_SS21_AD6=1;      //Set ADC function
/* Step 2. Enable and Select ADC clock source, and then enable ADC module */
SYSCLK->CLKSEL1.ADC_S = 2;        //Select 22Mhz for ADC
SYSCLK->CLKDIV.ADC_N = 1;         //ADC clock source = 22Mhz/2 =11Mhz;
SYSCLK->APBCLK.ADC_EN = 1;        //Enable clock source
ADC->ADCR.ADEN = 1;               //Enable ADC module
/* Step 3. Select Operation mode */
ADC->ADCR.DIFFEN = 0;             //single end input
ADC->ADCR.ADMD   = 0;             //single mode
/* Step 4. Select ADC channel */
ADC->ADCHER.CHEN = 0x80;          // channel bit [7:0]
/* Step 5. Enable ADC interrupt */
ADC->ADSR.ADF =1;                 //clear the A/D interrupt flags for safe
ADC->ADCR.ADIE = 1;
//NVIC_EnableIRQ(ADC_IRQn);
/* Step 6. Start ADC Conversion */
ADC->ADCR.ADST=1;
}
```



17

Smpl_ADC_VR1


```
int32_t main (void)
{
        char TEXT1[16]="ADC Value:     ";
        UNLOCKREG();
        SYSCLK->PWRCON.XTL12M_EN = 1; // enable external clock (12MHz)
        SYSCLK->CLKSEL0.HCLK_S = 0;          // select external clock (12MHz)
        LOCKREG();

        InitADC();                      // initialize ADC

        init_LCD();     // initialize LCD pannel
        clear_LCD();  // clear LCD panel
        print_Line(0, "Smpl_ADC_VR1");

        while(1)
        {
                while(ADC->ADSR.ADF==0); // wait till conversion flag = 1, conversion is done
                ADC->ADSR.ADF=1;            // write 1 to clear the flag
                sprintf(TEXT1+10,"%4d",ADC->ADDR[7].RSLT); // convert ADC7 value into text
                print_Line(1, TEXT1);       // output TEXT to LCD
                DrvSYS_Delay(20000);      // delay
                ADC->ADCR.ADST=1;      // restart ADC sample
        }
}
```

18

Smpl_ADC_IRQ

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

```
//
// Smpl_ADC_IRQ :
// sampling ADC0~7 continuously
// use Interrupt Callback function to set interrupt flag
// display ADC7 value on LCD when interrupt flag is set

// Note: ADC7 is connected to on-board Variable Resistor
//       rotating VR to change the display value
#include <stdio.h>
#include "NUC1xx.h"
#include "LCD_Driver.h"
#include "DrvGPIO.h"
#include "DrvSYS.h"
#include "DrvADC.h"

volatile uint8_t gu8AdcIntFlag;

void AdcIntCallback(uint32_t u32UserData)
{
   gu8AdcIntFlag = 1;
}
```

ADC Interrupt Callback function

Set ADC Interrupt Flag

23

Smpl_ADC_IRQ

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

```
int32_t main (void)
{
  int i;
  char TEXT[4][16];
  uint16_t adc_value[8];

  UNLOCKREG();
  SYSCLK->PWRCON.XTL12M_EN = 1;
  SYSCLK->CLKSEL0.HCLK_S = 0;
  LOCKREG();

  init_LCD();  // initialize LCD pannel
  clear_LCD();  // clear LCD panel
  print_Line(0,"Smpl_ADC_IRQ");

 // The maximum ADC clock rate is 16MHz
// open 8 ADC channels for continously sampling
```

select 12MHz as CPU clock source

Initialize LCD print project title

```
DrvADC_Open(ADC_SINGLE_END,
ADC_CONTINUOUS_OP, 0xFF, INTERNAL_HCLK, 1);
DrvADC_EnableADCInt(AdcIntCallback, 0);

i = 7;   // ADC channel = 7
gu8AdcIntFlag =0;      // reset AdcIntFlag
DrvADC_StartConvert(); // start ADC sampling

while(1) {
   // wait till    dcIntFlag is set by AdcIntCallback
   while(gu8AdcIntFlag==0);
   gu8AdcIntFlag=0; // reset AdcIntFlag
   // read ADC value from ADC registers
   adc_value[i] = DrvADC_GetConversionData(i);
    sprintf(TEXT[2],"ADC%1d:%4d",i,adc_value[i]);
    print_Line(2, TEXT[2]);
    }

   //DrvADC_Close();
}
```

sample 8 ADC channels continuoulsy

Enable ADC callback

read ADC value

24

Page 38

38

ADC with Nucleo Board



Page 39

39

Analog-to-Digital Conversion on the mbed

- The library functions are shown in the table below

AnalogIn	An analog input, used for reading the voltage on a pin
Functions	**Usage**
AnalogIn	Create an AnalogIn, connected to the specified pin
read	Read the input voltage, represented as a float in the range [0.0, 1.0]
read_u16	Read the input voltage, represented as an unsigned short in the range [0x0, 0xFFFF]
operator float	operator float An operator shorthand for read()

40

DISPLAYING ANALOG INPUT VALUE ON COMPUTER SCREEN

```
//Reads input through ADC, and transfers to PC terminal

#include "mbed.h"

Serial pc (USBTX,USBRX);   // Tx, Rx

AnalogIn Ain (PA_0);
//float ADCdata;
int main( )
{
      pc.printf ("ADC Data Values... \n\r") ;
      while (1)
      {
            //ADCdata = Ain ;
            pc.printf ("%f \n\r", Ain.read()) ;
            wait (0.5) ;
      }
}
```


41